

ERA Tutorium 11

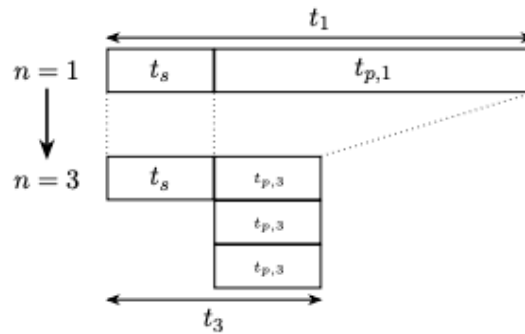
Leopold Cario	<u>Termin:</u>	<u>Raum:</u>	<u>Zulip:</u>
	· Mittwoch 10:00	03.13.010	ERA Tutorium - Mi - 1000 - 1
	· Donnerstag 12:00	00.08.059	ERA Tutorium - Do - 1200 - 1

Parallelisierung:

$$\cdot \text{Speedup}(n) = \frac{t_1}{t_n} \quad ; \quad t_n = t_s + t_{p,n}$$

Konstante Problemgröße:

$$\begin{aligned} \cdot \text{Speedup}_{\text{Amdahl}}(n) &= \frac{t_1}{t_s + \frac{t_{p,1}}{n}} \\ &= \frac{1}{s + \frac{(1-s)}{n}} \end{aligned}$$



$$\lim_{n \rightarrow \infty} \text{Speedup}_{\text{Amdahl}}(n) = \frac{1}{s + 0} = \frac{1}{s}$$

Proportionale Problemgröße:

$$\cdot t_1 = t_s + n \cdot t_p$$

$$\cdot t_n = t_s + \frac{n \cdot t_p}{n} = t_s + t_p$$

$$\begin{aligned} \cdot \text{Speedup}_{\text{Gustafson}}(n) &= \frac{t_1}{t_n} = \frac{t_s + n \cdot t_p}{t_s + t_p} \\ &= s + n \cdot (1-s) \end{aligned}$$

Lehrstuhl für
Rechnerarchitektur & Parallele Systeme
Prof. Dr. Martin Schulz
Dominic Prinz
Jakob Schäffeler

Lehrstuhl für
Design Automation
Prof. Dr.-Ing. Robert Wille
Stefan Engels
Benjamin Hien

Einführung in die Rechnerarchitektur

Wintersemester 2025/2026

Übungsblatt 11: Parallelisierung und Caches

12.01.2026 – 16.01.2026

1 Speedup

Wie viel Speedup kann man theoretisch bei den folgenden C-Programmen durch Parallelisierung bei Nutzung von 4 bzw. 16 CPU-Kernen erreichen?

- a) Verwenden Sie für die Berechnung des Speedups das Amdahlsche Gesetz: $S_{\text{Amdahl}} = \frac{T}{t_s + \frac{t_p}{n}}$

Halten Sie sich dabei an die folgenden Teilschritte:

- Ist das Programm parallelisierbar?
- Welche Zeilen können parallelisiert ausgeführt werden? Von wie vielen Threads maximal?
- Berechnen Sie jeweils die gesamte Laufzeit für parallelisierbare und seriell ausgeführte Programmteile.
- Nutzen Sie diese Werte zur Berechnung des Speedups.

Hinweis: Sie können von der (idealisierten) Annahme ausgehen, dass alle Zeilen, auch Schleifenstrukturen wie `for(...)` und `while(...)` bei jedem Schleifendurchlauf gleich viel Zeit benötigen. Die benötigte Zeit (für jede Iteration im Falle einer Schleife) ist hinter jeder Zeile angegeben.

- i) Das Zählen der Elemente in einer verketteten Liste.

```
struct item {  
    int data;  
    struct item* next;  
};  
  
int getCount(struct item* head)  
{  
    int count = 0;           // 1ns
```

```

struct item* cur = head; // 1ns
while (cur) {           // 3ns
    ++count;             // 2ns
    cur = cur->next;      // 15ns
}
return count;           // 1ns
}

```

$S = 1$

ii) Den Elementen eines Arrays einen Wert hinzufügen

```

float x[100];
...
void add_to_x(float base, float exponent) {
    float value = pow(base, exponent); // 100ns
    for(int i = 0; i < 100; i++) {     // 3ns 101x
        x[i] += value;                 // 17ns 100x
    }
}

```

$$t_s = 100ns$$

$$t_p = 17ns \cdot 100 + 3ns \cdot 101 = 2003ns \approx \underline{\underline{2000ns}}$$

$$S_{\text{Amdahl}}(4) = \frac{100ns + 2000ns}{100ns + \frac{2000ns}{4}} = \underline{\underline{3,5}}$$

$$S_{\text{Amdahl}}(16) = \frac{2100ns}{100ns + \frac{2000ns}{16}} \approx \underline{\underline{9,33}}$$

- b) (*Optional*) Das Gustafsonsche Gesetz ist eine Erweiterung des Amdahlschen Gesetzes. Allerdings wird im Unterschied dazu ein festes Zeitfenster betrachtet, und die Problemgröße wächst proportional zur Anzahl der verfügbaren Prozessoren.

Für die Berechnung des Speedups nach Gustafson betrachten wir ein Problem mit n -fachem parallelem Teil als Basis. Dieses kann auf einem Prozessor in $t_s + n \cdot t_p$ berechnet werden.

Auf n Prozessoren kann der parallel ausführbare Teil gleichmäßig auf alle Prozessoren aufgeteilt werden. Somit erhalten wir als Ausführungszeit für n Prozessoren

$$t_n = t_s + \frac{n \cdot t_p}{n} = t_s + t_p$$

Der Speedup, der auf n Prozessoren erreicht werden kann ist somit beschrieben durch:

$$S_{\text{Gustafson}}(n) = \frac{t_1}{t_p} = \frac{t_s + n \cdot t_p}{t_s + t_p} = \frac{t_s + n \cdot t_p}{T}$$

Oder mit $t_s + t_p$ normalisiert mit 1:

0,4 0,6

$$S_{\text{Gustafson}}(n) = \frac{\overset{S}{t_s} + n \cdot \overset{P}{t_p}}{1}$$

Berechnen Sie den Speedup nach Gustafson für die Programmteile aus Aufgabe a)

Was hat der Unterschied zu bedeuten?

$$S_{\text{Gustafson}}(4) = \frac{100\text{ns} + 4 \cdot 2000\text{ns}}{100\text{ns} + 2000\text{ns}} = \underline{3,86}$$

$$S_{\text{Gustafson}}(16) = \frac{100\text{ns} + 16 \cdot 2000\text{ns}}{100\text{ns} + 2000\text{ns}} = \underline{15,28}$$

2 Roofline Modell

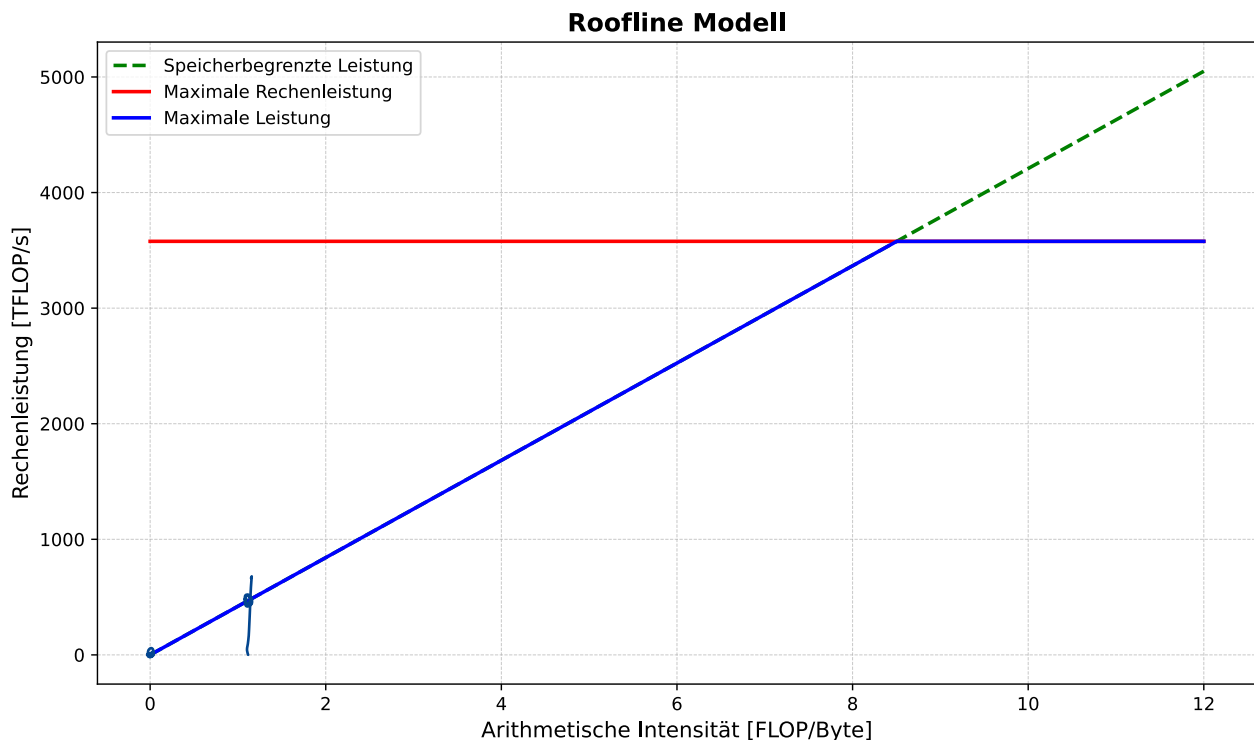


Abbildung 1: Roofline Modell

Verwenden Sie das in Abbildung 1 gegebene Roofline Modell, um die maximale Performance für die untenstehende Programme zu berechnen. Gehen Sie davon aus, dass alle Fließkommazahlen 32-Bit groß sind. Halten Sie sich dabei an folgendes Vorgehen:

- Bestimmen Sie die arithmetische Intensität I der wiederholten Durchführung der beschriebenen Operationen in FLOP/Byte. Bestimmen Sie hierfür zuerst wie viele Floatingpoint Operationen ausgeführt werden und wie viele Bytes gelesen und geschrieben werden. Benutzen Sie dann zur Berechnung von I folgende Formel:

$$I = \frac{W}{Q} = \frac{\text{Anzahl der Operationen in FLOP}}{\text{Benötigte Speichertransfers in Bytes}}$$

- Verwenden Sie Ihre Ergebnisse um aus dem Roofline-Diagramm die theoretisch maximale Leistung für die Operationen in FLOP/s abzulesen.
- Sind die Probleme rechen- oder bandbreitenlimitiert?

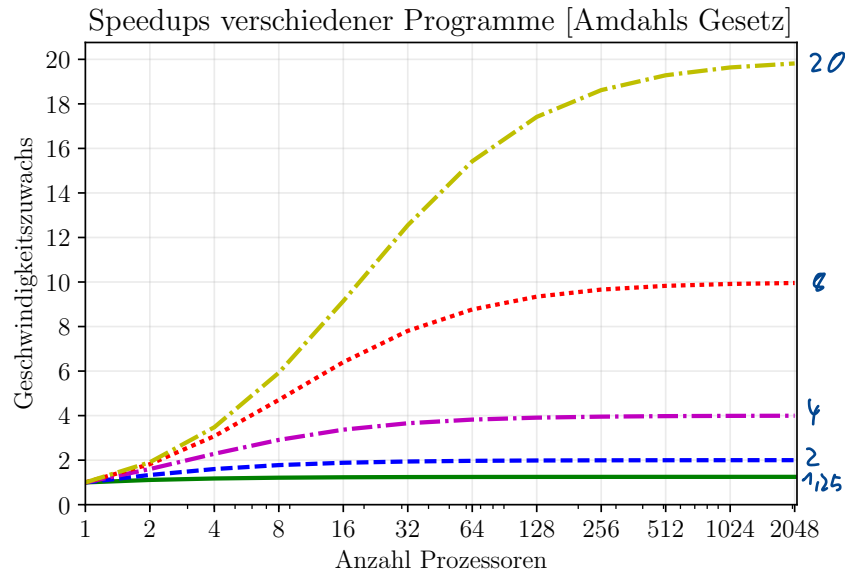
Programme

- String Copy:** Kopieren eines langen Strings aus dem Speicher an eine andere Speicherstelle. $W=0$ $I=0$
- Potenzieren von Zahlen:** Laden eines Arrays aus Fließkommazahlen aus dem Speicher, 10-fache Fließkomma-Multiplikation der Elemente mit sich selbst (also insgesamt 9 Multiplikationen), abspeichern an der ursprünglichen Speicherstelle.

$$I = \frac{W}{Q} = \frac{9 \text{ FLOPs}}{2 \cdot 4 \text{ B}} = 1,125 \text{ FLOPs/Byte}$$

3 Auswertung von Speedupdiagrammen

Sie organisieren in einem Rechenzentrum die Ressourcenverteilung und bekommen untenstehendes Speedupdiagramm für fünf Programme vorgelegt. Beobachten Sie die Kurve und diskutieren Sie die Abwägungen bei der Auswahl der Anzahl der Prozessoren für jedes Programm. Beobachten Sie den maximalen Speedup und begründen Sie dessen Ursprung.



$$\lim_{n \rightarrow \infty} S_A(n) = \frac{1}{S}$$

Abbildung 2: Speedup nach Amdahls Gesetz fünf verschiedener Programme

- a) Wie groß ist der parallelisierbare Anteil der fünf Programme?

Gelb: $20 = \frac{1}{S} \Rightarrow S = \frac{1}{20}$ $p = 1 - \frac{1}{20} = \frac{19}{20} = 95\%$

Rot: $S = \frac{1}{10}$ 90%

Lila: $S = \frac{1}{4}$ 75%

Blau: $S = \frac{1}{2}$ 50%

Grün: $S = \frac{1}{1.25}$ $p = \frac{0.125}{1.25} = 10\%$

- b) Das in Abbildung 3 dargestellte Diagramm zeigt den berechneten Speedup eines Programms und den real gemessenen. Wie kommt der sichtbare Unterschied zustande?

Lösungsvorschlag

Sub-linearer Speedup. Diverse Gründe sind möglich, z. B. dass sich die Threads begrenzte Ressourcen teilen müssen (Speicherbandbreite), oder für Threaderstellung und Kommunikation zusätzlich Zeit benötigt wird.

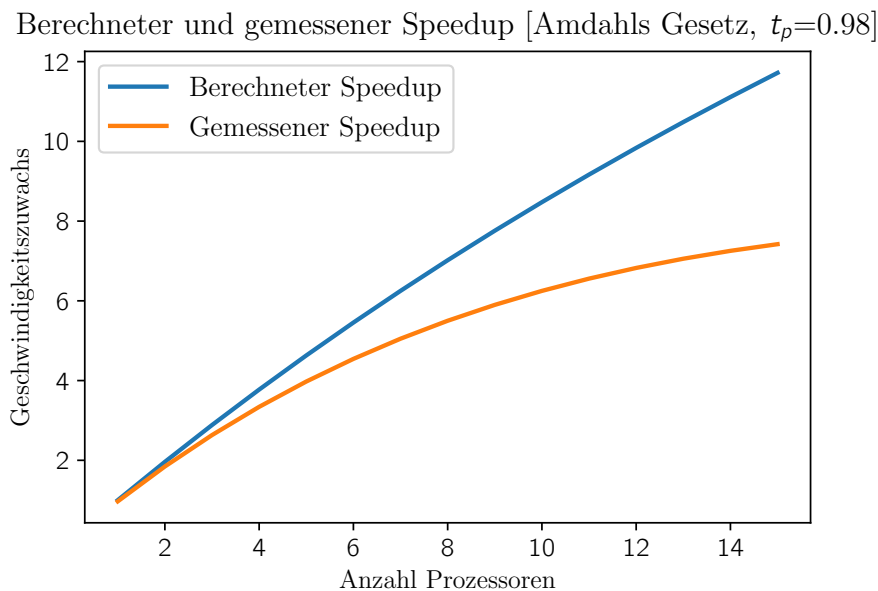


Abbildung 3: Unterschied gemessener und berechneter Speedup, Beispiel 1

- c) Das in Abbildung 4 dargestellte Diagramm zeigt den berechneten Speedup für ein anderes Programm für große Datenmengen und den real gemessenen. Wie kommt der sichtbare Unterschied zustande?

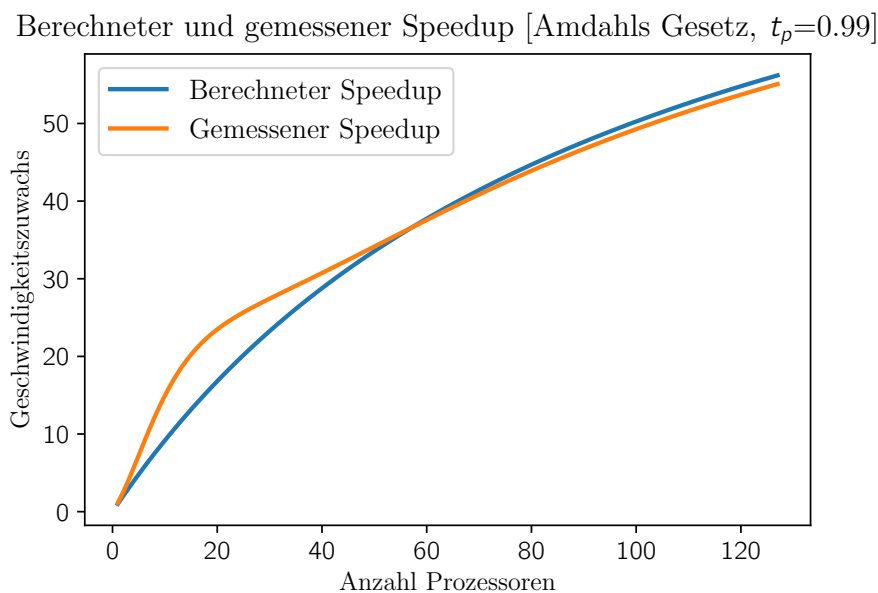


Abbildung 4: Unterschied gemessener und berechneter Speedup, Beispiel 2

Teilweise superlinearer Speedup. Problem/Bottleneck sind die Datenmengen (vgl. Roofline-Diagramm), Mehr CPUs $\rightarrow$ mehr Cache-Speicherplatz $\rightarrow$ schnellerer Programmablauf.

- d) Was kann man generell anhand eines Speedupdiagramm über die Rechenleistung eines Programms aussagen? Was nicht?

Speedup sagt aus, wie viel schneller ein Programmcode ist, wenn parallelisiert ausgeführt wird. Er sagt nichts über Effizienz des Algorithmus oder Programmcodes, Programmdauer etc. aus.

MESI

Sie haben in der Vorlesung das MESI-Cachekohärenzprotokoll kennengelernt. In dieser Hausaufgabe simulieren Sie die Zustandsübergänge für ein MESI-System mit vier Prozessoren, die jeweils über einen eigenen, **direktabbildenden** Cache verfügen. Die Caches bestehen jeweils aus **vier Cachezeilen zu je acht Bytes**.

Alle Speicherzugriffe sind nur ein Byte groß. Sie können also in dieser Hausaufgabe (wieder) vernachlässigen, dass sich ein (unausgerichteter) Speicherzugriff auch über mehrere Blöcke im Speicher erstrecken kann.

Vorüberlegung (nicht abzugeben)

Überlegen Sie sich, welche Bits der Adressen **Offset-, Index- und Tagbits** sind.

Offset: $\log_2(8) = 3$
 Index: $\log_2(4) = 2$
 Tag:

⦿ MESI-Flags Keine Ergebnisse

Simulieren Sie nun die durch Speicherzugriffe verursachten MESI-Zustandsübergänge. Die Speicherzugriffe werden durch die ersten drei Spalten der Tabelle in der Datei **submission.txt** beschrieben: **Prozessor** gibt an, welcher Prozessor auf den Speicher zugreift, **R/W** spezifiziert, ob es sich um einen Lese- (**R**) oder Schreibzugriff (**W**) handelt, und **Adresse** gibt die Adresse der Speicherzelle an, auf die zugegriffen wird.

Die letzten vier Spalten enthalten die MESI-Zustände der vier Prozessoren *nach* dem jeweiligen Speicherzugriff. Die MESI-Zustände werden dabei für jeden Prozessor als 4-er Tupel aus den Ziffern **M**, **E**, **S** und **I** codiert. Dabei codiert jeweils die Ziffer ganz links den Zustand der Cache-Zeile mit Index 0.

Der Anfang der Tabelle ist bereits ausgefüllt. Vervollständigen Sie die Tabelle.

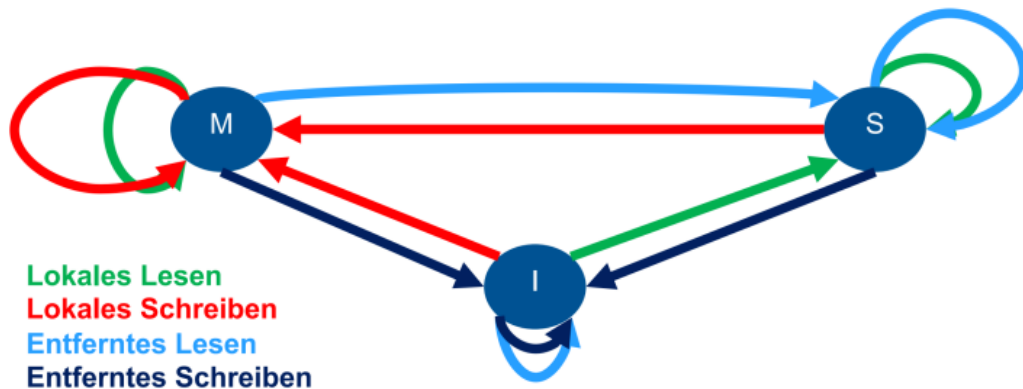
# Füllen Sie die Tabelle so aus, dass sie die MESI-Flags nach den entsprechenden Speicherzugriffen zeigt.								
#	Zugriff			MESI-Zustände				
	Prozessor	R/W	Adresse	P0	P1	P2	P3	
6	--	-	---	IIII	IIII	IIII	IIII	
7	P0	R	11010000	II EI	IIII	IIII	IIII	
8	P2	R	01010010	II EI	IIII	II EI	IIII	
9	P1	R	11010100	II EI	II EI	II EI	IIII	
10	P2	W	11010000					
11	P3	W	01010000					
12	P3	R	01010001					
13	P0	R	11111111					
14	P1	R	01010101					
15	P0	R	11111111					
16	P3	R	11111000					
17	P3	W	11011000					
18	P3	W	11000000					

Cache-Kohärenz: Probleme



ERA – Zentralübung #10: Caches & Parallelität

MSI Protokoll



MESI Protokoll

